

TrustCore on the ZCU106

Vivado 2025.1 — manual block-design bring-up guide

Vivado version	2025.1 (SW build 6140274)
Target board	Xilinx ZCU106 evaluation board
FPGA part	xczu7ev-ffvc1156-2-e
Sources	~/Desktop/trustcore_vivado
Repository	github.com/tk685-cpu/trustcore
Design	SHA-256 / Ascon-128 crypto chip, AXI4-Lite wrapped

This is the manual route. There is also a scripted build (`scripts/build_zcu106.tcl`) that does all of section 2 in one command, but it has never been executed against a real Vivado install, so the steps below are the reliable path.

0. Before you start

All sources are already on this machine at `~/Desktop/trustcore_vivado`. Nothing needs downloading. The three directories that matter:

- `rtl/` — the ASIC deliverable. Instantiated unmodified; do not edit.
- `rtl_fpga/` — FPGA-only test scaffolding. Not intended to tape out.
- `constraints/zcu106_crypto.xdc` — pin assignments for LEDs and buttons.

1. Create the project

1. Launch Vivado 2025.1. Choose **Create Project** → RTL Project, and **uncheck** “Do not specify sources at this time”.
2. Add sources: every `.v` file from **both** `rtl/` and `rtl_fpga/`. Leave “Copy sources into project” **unchecked** so edits on disk stay in sync.
3. Add constraints: `constraints/zcu106_crypto.xdc`.
4. Select the part `xczu7ev-ffvc1156-2-e`, or pick the ZCU106 board from the Boards tab if it is installed.
5. Confirm in Sources → Hierarchy that `crypto_axi_top` is the top module and that `button_ctrl` appears beneath it with no black box.

2. Build the block design

You add the Zynq UltraScale+ MPSoC exactly ONCE. Steps 2 and 3 below both talk about that block, but step 3 is *configuring the block you already placed* — double-clicking it to open its settings. It is not a second block. If you end up with both `zynq_ultra_ps_e_0` and `zynq_ultra_ps_e_1` on the canvas, delete the second one: the AXI connection automation in step 5 assumes a single PS.

1. IP Integrator → **Create Block Design**. Accept the default name.
2. Click + (Add IP) and add **Zynq UltraScale+ MPSoC**. When the green banner appears, click **Run Block Automation** and apply the board preset.

3. **Double-click that same PS block.** Enable **PL Clock 0 at 100 MHz** and the `M_AXI_HPM0_LPD` master port. Everything else stays at the preset.
4. Right-click empty canvas → **Add Module** → `crypto_axi_top`. Vivado infers the AXI4-Lite interface from the `s_axi_*` port names.
5. Run **Connection Automation** on `crypto_0/s_axi`. This inserts the AXI interconnect and the processor reset block, and wires the clocks.
6. Right-click the `led` and `btn` pins → **Make External**. Then **Address Editor** → **Assign All**. Validate Design, create the HDL wrapper, and generate the bitstream.

Make External renames your ports. Vivado usually appends an instance suffix, so `led` becomes `led_0` and `btn` becomes `btn_0`. The build then fails with `No objects matched 'get_ports led'`. Fix it in the block design, not the XDC: click the external port and set **Name** back in the External Port Properties panel. Also confirm `led` came out as **one 8-bit port** `led[7:0]`, not eight scalars.

Verify the port names before you synthesize

Open the synthesized design and run this in the Tcl console:

```
get_ports *
```

The list must contain exactly `led[0]..led[7]`, `btn[0]` and `btn[1]`. Note the base address from the Address Editor — you need it in Vitis.

3. Vitis application

1. Export hardware **including the bitstream**. This produces an XSA.
2. New Application Project on that XSA, processor `psu_cortexa53_0`, standalone BSP, Empty Application (C).
3. Copy `vitis/main.c` and `vitis/crypto_vectors.h` into `src/`.
4. If the ID register read fails, set `CRYPTO_BASE` in `main.c` to the address you noted from the Address Editor.
5. Program the FPGA, run, and watch the UART at **115200 baud**.

4. Bring-up order

Do not try to light the whole thing up at once.

Step	What to do	Why it matters
1	Read the ID register. It returns <code>0x5AA5C0DE</code> .	The single most useful check. If it reads back correctly, the bitstream, clock, reset, address map and AXI path all work, so any later failure is logic, not infrastructure. If it reads <code>0x00000000</code> or <code>0xFFFFFFFF</code> , stop and fix that first.
2	Run the SHA vectors.	No key or nonce needed, so this is the simplest end-to-end path.
3	Run the Ascon vectors.	If the ciphertext matches but the tag does not, you are running the old core.
4	Negative tests.	Bad command and oversize length must be rejected, and the chip must still answer correctly afterwards.

If something stalls, put an ILA on `data_ready`, `cmd_sha256`, `cmd_ascon`, the `spi_buffer_ctrl` state register, `byte_idx`, `result_valid` and `busy`. On a ZU7EV that costs almost nothing and shows exactly which state it

stalled in.

5. Pushbutton control

Press a button on the ZCU106 5-way switch to run a test suite — no laptop needed once the bitstream and application are loaded.

Button	Action
GPIO_SW_W	Ascon-128 vectors
GPIO_SW_E	SHA-256 vectors
both within 250 ms	everything, plus rejection and recovery checks

Do not use SW3 or SW4. On the ZCU106 those are SRST_B and POR_B: they reset the MPSoC and are not routed to the PL at all. The user pushbuttons are SW14–SW18, the 5-way navigation switch, which appear in the master XDC as GPIO_SW_N/S/E/W/C.

Pins are already filled in from the Rev1.0 master XDC: GPIO_SW_W is AK12 and GPIO_SW_E is AC14, both bank 66 at LVCMOS12. On a different board revision, re-check them against that revision's master file.

6. Updating an existing Vivado project

Copying the new files over the old ones is **not sufficient**, because the top-level port list changed.

1. rtl_fpga/button_ctrl.v is a **new file** — replacing files does not add it. Add Sources, select it, leave “Copy sources into project” unchecked. Check that button_ctrl then appears under crypto_axi_top with no black box.
2. crypto_axi_top gained a btn port, and led_pass / led_fail were replaced by a single 8-bit led port. A block design cell caches the module's port list, so right-click the crypto_0 cell → **Refresh Module**. If the cell stays in an error state, delete it, re-add via Add Module, and re-run connection automation on s_axi.
3. Delete the old led_pass and led_fail external ports. They have nothing left to connect to and will cause a validation error.
4. Make btn and led external, then re-check the port names against the warning in section 2.

Then Validate Design, regenerate the HDL wrapper if prompted, and rebuild. In Vitis, re-import the XSA so xparameters.h matches, then rebuild the application.

If any of that turns messy, deleting the project and rebuilding from section 1 is faster than untangling it. Nothing in the project cannot be rebuilt in five minutes.

7. Scripted alternative

Everything in section 2 in one command:

```
cd ~/Desktop/trustcore_vivado/scripts
vivado -mode batch -source build_zcu106.tcl
```

The Tcl is written against the standard API and resolves IP versions dynamically, but it has not been run against a real Vivado install. Treat it as a starting point; if it fails, fall back to the manual steps.